

[Marcar como hecha](#)

Inicializando y paquete de trabajo Node.js

Si desea crear un nuevo paquete, puede crear el archivo package.json a mano o puede obtener la ayuda de npm. El comando `npm init` te lleva a través de un pequeño diálogo para obtener valores iniciales para el archivo package.json.

Una vez que pasas las preguntas, el archivo package.json se escribe en el disco. Espere tener que editar ese archivo considerablemente antes de publicarlo en el repositorio npm. Algunos campos ayudan a dar una buena impresión a la gente que mira el listado de paquetes en npmjs.com:

- Enlace a la página de inicio, y URL de la cola de emisión
- Palabras clave, para que pueda ser enlazado con otros paquetes similares
- Una buena descripción que ayude a la gente a entender el propósito

La versión de node debe estar declarada

This dependency is declared in package.json using the engines tag:

```
"engines": {  
  "node": ">= 8.x"  
}
```

Dentro de la carpeta `Node_pruebas01` veréis bastantes muestras de lo que hemos visto hasta aquí

Dentro de la carpeta `Node_pruebas02` veréis bastantes nuevas cosas, y lo que veremos desde aquí

Tenéis una carpeta `12events` donde se trabaja con eventos de node.js.

`pulser.js` se ocupa de enviar y recibir "pulsos" mientras que `pulsed.js` va instanciando objetos `Pulser`, que es lo que el otro módulo o documento exporta (lo que cualquier módulo o doc de node exporte es lo único que será accesible a otros documentos. (require))

La primera aplicación que en 6 líneas de código levantaba un server y nos mostraba el `helloWorld`, creaba un objeto `http server`, que es un `EventEmitter`. De modo que se entienda mejor, podría utilizarse un `server.on` donde `on` es una función que consume un `send`

```
const http = require('http');  
const server = http.createServer();  
server.on('request', (req, res) => {  
  res.writeHead(200, {'Content-Type': 'text/plain'});  
  res.end('Hello, World!\n');  
});  
server.listen(8124, '127.0.0.1');  
console.log('Server running at http://127.0.0.1:8124');
```

estas pequeñas apps se ejecutan con node y el nombre con extensión del archivo, pero en cuanto la aplicación se hace un poco mayor, es más cómodo colocar un `start` en el `package.json` (una ruta de entrada a la aplicación). El módulo **nodemon**, que se instala por npm como cualquier otro, reinicia la aplicación cada vez que guardamos cambios, por lo que es muy cómodo. Esto permite ejecutarla con `npm start`. Siempre y cuando antes en el mismo `package.json`, dentro de los scripts incluyamos `"start": "nodemon index.js"` en caso de que `index` sea el nombre de nuestro archivo de arranque.

En general las aplicaciones se van montando mientras se instalan componentes, cada componente queda registrado en el `package.json`, por lo que la carpeta `node_modules` no es necesaria, ya que de npm se descargan los paquetes allí declarados. Esto se hace con `npm install`. Esto ocurre en la medida que los paquetes no se han instalado de modo global en los equipos, en este caso, no quedan registrados en `package.json` y no cumple las normas de los 12 factores

Framework Express

Entre las razones más importantes por las que se utiliza un framework en node están:

?

- Uso de cookies de modo que se mantengan sesiones
- autenticaciones
- manejo de filas estáticas
- seguridades, límites, páginas de error, etc

Express es quizás el framework de aplicaciones web Node.js más popular. Es tan popular que es parte del acrónimo MEAN Stack. MEAN se refiere a MongoDB, ExpressJS, AngularJS y Node.js. O MEARN si se utiliza react

la instalación es mediante

```
npx install express-generator
```

pudiendo añadir versión después de una @, por ejemplo

```
npx install express-generator@4.x
```

Recomendable la página <http://expressjs.com/>

una vez instalado, es posible ejecutar express --help, y veremos, entre otras cosas, las diferentes plantillas de vistas como .hbs, .jade o .pug, las de css (ver abajo) y otros comandos

```
Usage: express [options] [dir]
Options:
-h, --help      output usage information
-V, --version    output the version number
-e, --ejs       add ejs engine support (defaults to jade)
--hbs          add handlebars engine support
-H, --hogan     add hogan.js engine support
-c, --css <engine> add stylesheet <engine> support (less|stylus|compass|sass) (defaults to plain css)
--git          add .gitignore
-f, --force     force on non-empty directory
```

Como veréis en la carpeta 21express del paquete dentro de **Node_pruebas02**

express.generator crea un app.js, un package.json y las carpetas: bin, public, routes y views

También automatiza el start en el package.json, con **npm start**

veamos en detalle cada cosa:

El directorio **views** contiene dos archivos, error.hbs e index.hbs. La extensión hbs se utiliza para los archivos Handlebars. Otro archivo, layout.hbs, es el diseño de página por defecto. Handlebars tiene varias formas de configurar plantillas de diseño e incluso parciales (fragmentos de código que pueden incluirse en cualquier lugar). Recordemos que handlebars es solo uno de los motores de vistas, hoy son más comunes .jade, que ahora es .pug, incluso twig, el de symfony pero utilizamos estas en el ejemplo.

El directorio **routes** contiene la configuración inicial del enrutamiento, es decir, el código para manejar determinadas URL específicas.

El directorio **public** contendrá activos que la aplicación no genera, sino que son simplemente se envían al navegador. Lo que se instala inicialmente es un archivo CSS, public/stylesheet/style.css.

El archivo **package.json** contiene nuestras dependencias y otros metadatos.

El directorio **bin** contiene el script www que vimos antes. Es un script Node.js, que inicializa los objetos HTTPServer, lo inicia escuchando en un puerto TCP, y llama al último archivo que discutiremos, **app.js**. Estos scripts inician Express, conectan los módulos de enrutamiento y hacen otras cosas.

si queremos utilizar otro motor de vistas especificamos con express --view=pug [nombreApp]

El que viene por defecto ahora es moustache

Apéndice

El siguiente código muestra la cantidad de información, parecida al sysinfo de php, que se puede obtener por medio del servidor. Usadla con cuidado, porque entre otras cosas muestra vuestra ip pública

server.js

```
const http = require('http');
const util = require('util');
const url = require('url');
const os
  = require('os');
const server = http.createServer();
server.on('request', (req, res) => {
  var requrl = url.parse(req.url, true);
```

```
  if (requrl.pathname === '/') {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.end(
      `<head><title>Hello, world!</title></head>
<body><h1>Hello, world!</h1>
<p><a href='/osinfo'>OS Info</a></p>
</body></html>`);
  } else if (requrl.pathname === "/osinfo") {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.end(
      `<head><title>Operating System Info</title></head>
<body><h1>Operating System Info</h1>
<table>
<tr><th>TMP Dir</th><td>${os.tmpdir()}</td></tr>
<tr><th>Host Name</th><td>${os.hostname()}</td></tr>
<tr><th>OS Type</th><td>${os.type()} ${os.platform()} ${os.arch()}
${os.release()}</td></tr>
<tr><th>Uptime</th><td>${os.uptime()}
${util.inspect(os.loadavg())}</td></tr>
<tr><th>Memory</th><td>total: ${os.totalmem()} free:
${os.freemem()}</td></tr>
<tr><th>CPU's</th><td><pre>${util.inspect(os.cpus())}</pre></td></tr>
<tr><th>Network</th><td><pre>${util.inspect(os.networkInterfaces())}</pre>
</td></tr>
</table>
</body></html>`);
  } else {
    res.writeHead(404, {'Content-Type': 'text/plain'});
    res.end("bad URL " + req.url);
  }
});
server.listen(8124);
console.log('listening to http://localhost:8124');
```